

Lab #4: Full-Adder Subtractor Calculator

In this lab, you will learn about full adders and perform two's complement binary arithmetic in software.

This lab is to be done with your lab partner only, and one lab report will be submitted per team.

Introduction: The Full Adder

For the last thousands of years, all mathematical calculations were performed by hand. Trigonometry, square roots, integrals, exponentials, logarithms, etc., were reduced to approximations using nothing but simple addition, subtraction, multiplication, and division. First, standalone astronomers or sailors needed to estimate orbits or headings. Later, rooms of people worked together to perform calculations that guided rockets into space. As you can guess, this was expensive, time-consuming, and error-prone.

Today, in stark contrast, one single CPU core inside your portable laptop, running at multiple gigahertz, can do more calculations *in one second* than a human can do *in their entire life*, even if they work *24 hours per day*. That doesn't even include the other CPU cores or the graphics processor. How did we get here?

While we will not delve into the complexities of computer technology since the 1940s, we will look at the fundamental building block that allows computers to do arithmetic: the full adder. Recall that when adding a binary number, every individual addition requires three pieces of information: bit A_n , bit B_n , and carry bit C_n . The carry bit comes from a previous step that may have carried over. Given these three inputs, the full adder circuit is simply designed to give the correct sum bit S_n and carry bit C_{n+1} . If full adders are connected in series, a sum with an arbitrary number of bits can be computed effortlessly. With a small modification, this circuit can also be used to perform subtraction. With addition and subtraction, multiplication and division may be performed, allowing any approximation to be made. This lab will focus on the full adder and the modification required to perform subtraction.

Part 1: Using K-Maps to Solve Full-Adder Truth Table

For this first part, you will study the full adder circuit's truth table. Find the truth table given in the Lab 4 module and, using K-maps, solve for the two functions that define the full adder. Your work may be either typed or handwritten.

After solving for the Cout (carry-out) and Sum functions, you will build both of them in either SimUAid or LTspice. Using the methods you learned in Lab 2, you will acquire a timing diagram that cycles through the entire truth table, showing all combinations.

Note: For SimUAid, add probes to your inputs and outputs, and label them as well. For LTspice, label your inputs and outputs.

Deliverables/Questions:

1. Copy the truth table for the full adder (Cout and Sum) into your lab document.
2. What does Sum mean in the context of binary addition?
3. What does Cout mean in the context of binary addition?
4. Provide the simplified **sum-of-products** formulas for Sum and Cout. Show your K-maps.
5. Provide the simplified **product-of-sums** formulas for Sum and Cout. Show your K-maps.
6. How many minterms and maxterms do the Cout and Sum formulas each contain?
7. Build the Cout and Sum circuits in one simulation file. Provide a screenshot of your schematics and the timing diagram for all combinations of A, B, and C. Label your inputs and outputs as necessary to label the waveforms on the timing diagram.
8. Research what an “ALU” is. What does it have to do with the full adder?

Part 2: Full-Adder Macro Chip Introduction

In a simulator, a macro is a component that replaces a set of components to make the circuit cleaner and more scalable. In SimUAid, there is a macro for the full adder function, which replaces all the gates you used in Part 1 with a single block with all necessary inputs and outputs. You can find this component via

Part => Other Parts => Full-Adder

In the same file that you completed Part 1, build the Full-Adder using this new component. Make sure it has the same inputs as those from Part 1, and compare both versions.

LTspice does not have a full adder macro pre-installed. Instead, you will have to create a *user-defined component* from your schematics in Part 1. Search for how to do this online and follow the instructions. It is recommended to copy your Part 1 file so that you may make edits without ruining it. Once complete, you will have a full adder as a single unit. Alternatively, you may find an online digital circuit simulator that has a full adder.

In a new simulation file:

You will build a 8-bit calculator using only the full adder macro—no ANDs or XORs. You should do this by attaching each macro in the manner as shown in Figure 4-3, which can also be found in the Lab 4 module. The only difference will be that your last Cout **will not** wrap back to C0 of your first Full-Adder. It will be an output. Additionally, for this build, you will go back to using switches if using SimUAid. For LTspice, you will either must program various combinations or you may find an online digital circuit simulator that has switches.

Note: For your 8-bit addition calculator, all inputs are treated as two’s-complement values.

Note: You will have 17 inputs in this build. A input will consist of A7 to A0, B input will consist of B7 to B0, and C0 will also be an input. C0 will also be referred to as Cin. With both A7 and B7 being the most significant bits (MSB) and A0 and B0 as the least significant bits (LSB).

Deliverables/Questions:

9. Build the 8-bit adder described above using full adder macros.
10. What are the major differences between your Part 1 build and the full adder macro? Why might you choose to use one over the other?
11. What is the difference between *unsigned* overflow and *signed* overflow?
12. Give **four** examples of $A + B$, where A and B can be two's complement positive or negative values. Include screenshots of your SimUAid / LTspice / online build doing these examples in your report. Caption your figures to indicate what numbers are being added and what the result is.

Part 3: Full Adder-Subtractor

For this last part of the lab, you will upgrade your 8-bit full adder calculator to do subtraction as well. This will be possible by adding XOR gates to your design.

The XOR gates (which always toggle their output when *one* input changes) will perform **one's complement** to B. To get the +1 required to have the **two's complement** of B, C_0 will be toggled to add 1 to the lowest bit in the sum. This is all equivalent to adding A to the two's complement of B, and it can be toggled on and off via one control bit, Cin.

Referring back to Figure 4-3, you will add XOR gates to all of your B inputs. Y_n will be the result of the original B_n input value and Cin (which is labeled as C_0 in Figure 4-3). This will two's-complement B, thereby making subtraction possible. An additional figure is in the Lab 4 module showing how this is achieved using a 2-bit adder-subtractor.

Note: You are still treating your inputs as two's-complement values.

Deliverables/Questions:

13. In your own words, explain the benefit of adding the XOR gate into your Part 2 circuit build.
14. What are the limits of your adder-subtractor calculator? With 8 bits, what are the largest and smallest permissible two's complement numbers that it can handle before undergoing *signed overflow*?
15. Give **four** examples of $A - B$, where A and B can be two's complement positive or negative values. Include screenshots of your SimUAid / LTspice / online build doing these examples in your report. Caption your figures to indicate what numbers are being added and what the result is.

Part 4: Summary of Deliverables

Turn in your report in either PDF or Word format, and make sure it is named:

ECE3441_Team#_Lab4

****Note: Please refrain from using notebook paper or writing in the attached lab. Make a separate Word document and have it submitted as a word or as a pdf document in BB.**

Points will be deducted if you do not follow instructions.

Use the following structure for your lab report:

Course Name and Number – Semester

Lab Number and Title

Names: Lab Partner(s)

Overview: Goals of the Lab

Briefly (~ 1 small paragraph) outline the goal of this lab, as well as the work done.

Step-by-Step Procedures and Results

This must be in chronological order of how Lab 4 should be completed. Make sure you include all of the necessary information that is mentioned in the lab and answer all questions.

Conclusion

Summarize what you have learned in this lab.